Lab #7: Meet the SPA's JS Frameworks

Anthony Bayate | Web Client-Side Programming | Kean University | October 2026

Setup

Each framework was scaffolded with *npm create vite@latest <name> -- --template <framework>* (Vite 8, JavaScript variants), followed by *npm install* and *npm run dev*. Vue and Preact also got *npm install materialize-css*; the others load Materialize from the CDN via *@import*. React was skipped as the handout says. For every framework there are three screenshots: the default template, the template with my name added, and the finished customized app.

Problems found while running the handout code

- **Preact:** the Met's */v1/search* endpoint was retired on 2026-10-01 (HTTP 410); moved to */v1.1/search*.
- **Lit:** the Tenor GIF API is discontinued (HTTP 403); replaced with DummyJSON fashion products.
- **Svelte:** *new App()* does not work in Svelte 5; *mount()* is required.
- **Qwik:** not offered by create-vite any more; *useClientEffect$* and the *innerHTML* prop no longer exist.

Task #1. Vanilla JavaScript: TV Show Finder (TVMaze API)

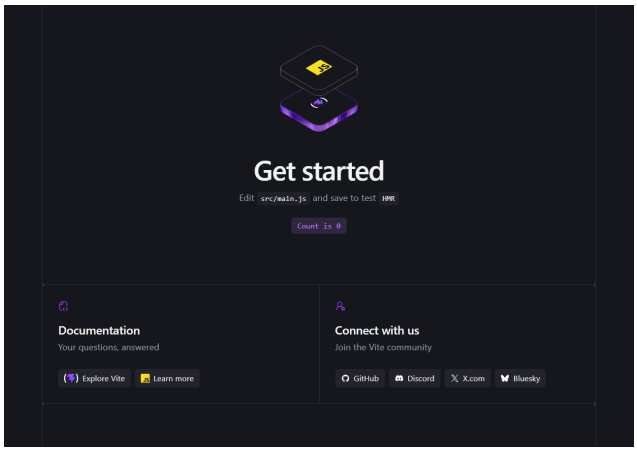
Project folder: *vanilla/* Dev server: *http://localhost:5101/*

Code discussion (with LLM help)

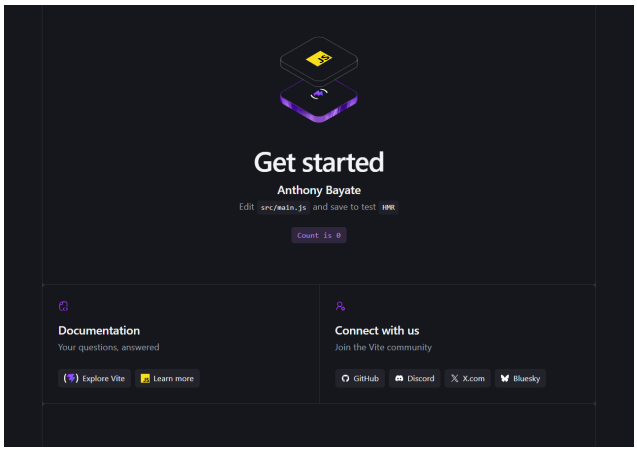
The starter waits for *DOMContentLoaded*, fetches */shows* from TVMaze, and appends one Materialize card per show with *innerHTML* +=. Discussing it with an LLM pointed out four problems: the loading bar and error div were never used; *movie.image.medium* crashes when a show has no image; the summary is raw HTML from a third party, injected straight into the page; and *innerHTML* += inside a loop re-parses the whole container every iteration.

Customizations and improvements

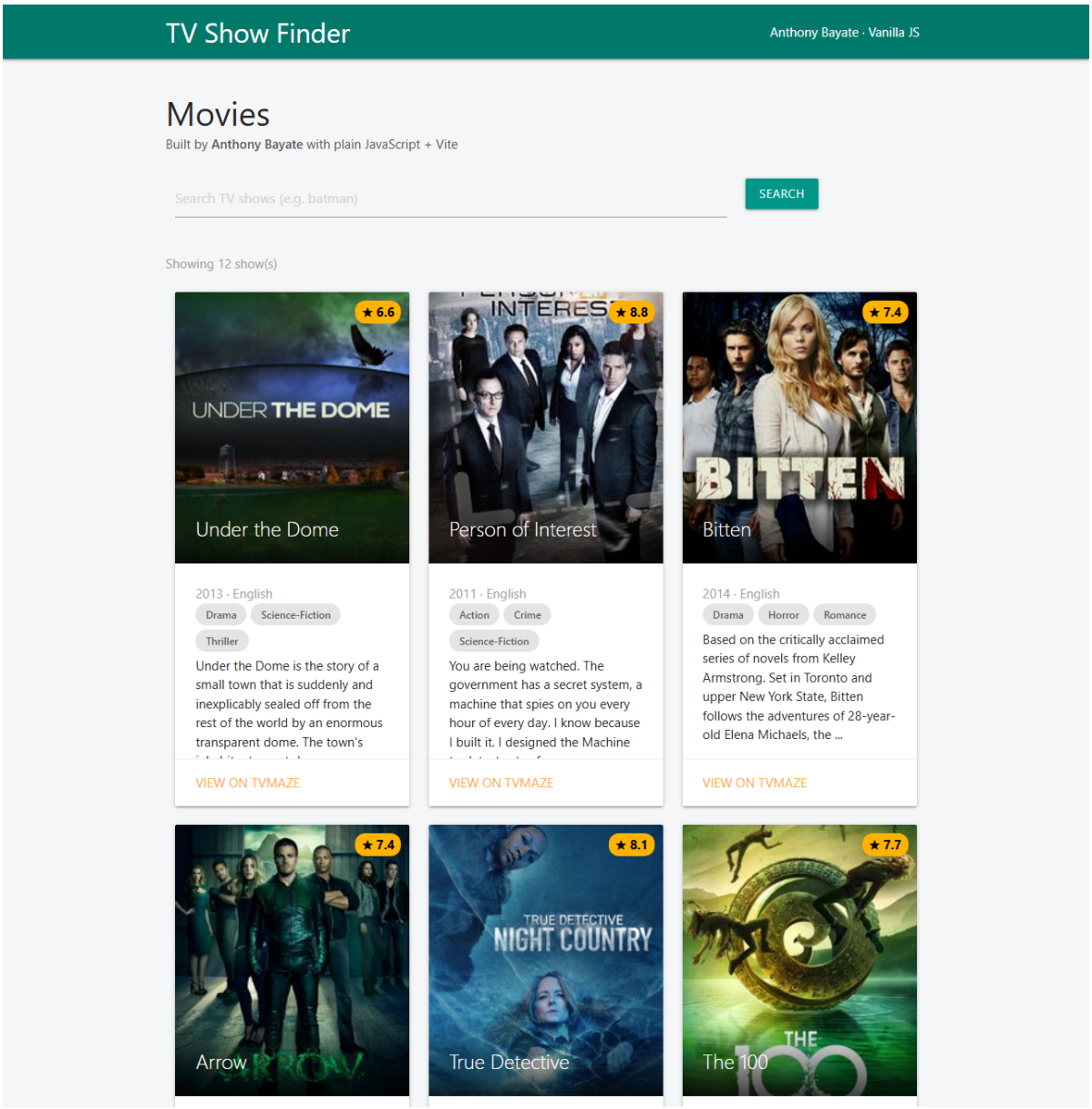
- Search form that calls TVMaze's */search/shows?q=* endpoint (results are unwrapped from *{score, show}*).
- Loading bar and error panel are shown/hidden in *try/catch/finally*; non-200 responses throw.
- Summaries are stripped to plain text with *DOMParser*, and names are escaped before rendering.
- Placeholder image when *image* is null, lazy-loaded images, rating badge, genre chips, premiere year.
- All cards are built with *map().join()* and written to the DOM once.



Default Vanilla JavaScript template from Vite



Template with my name added to the GUI



Final customized app (Vanilla JavaScript) with my name

Task #2. Vue 3: Album Explorer (iTunes Search API)

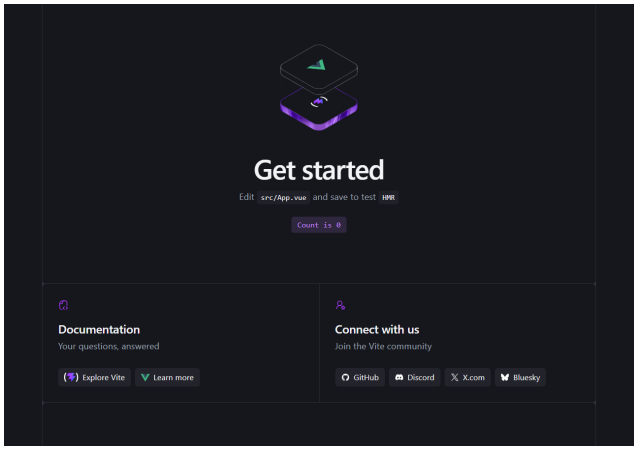
Project folder: *vue/* Dev server: *http:///localhost:5102/*

Code discussion (with LLM help)

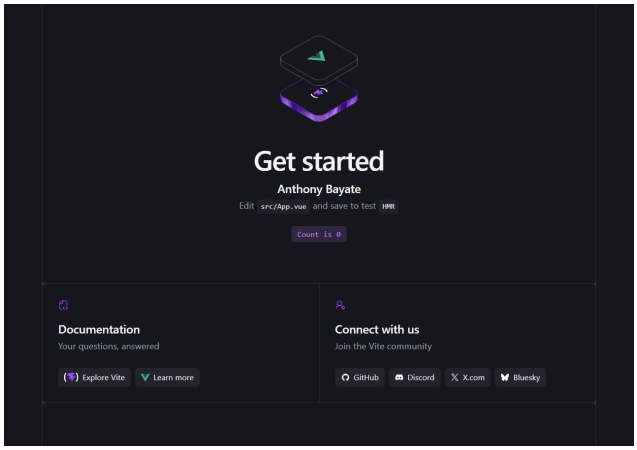
The component uses the Composition API: *ref()* for albums, error and loading, and *onMounted* to fetch once. *v-if* / *v-else-if* / *v-else* switch between error, spinner and grid, and *v-for* with *:key* renders the cards. The LLM noted the search term was hard-coded, the 100x100 artwork looks blurry when stretched, and the response status was never checked.

Customizations and improvements

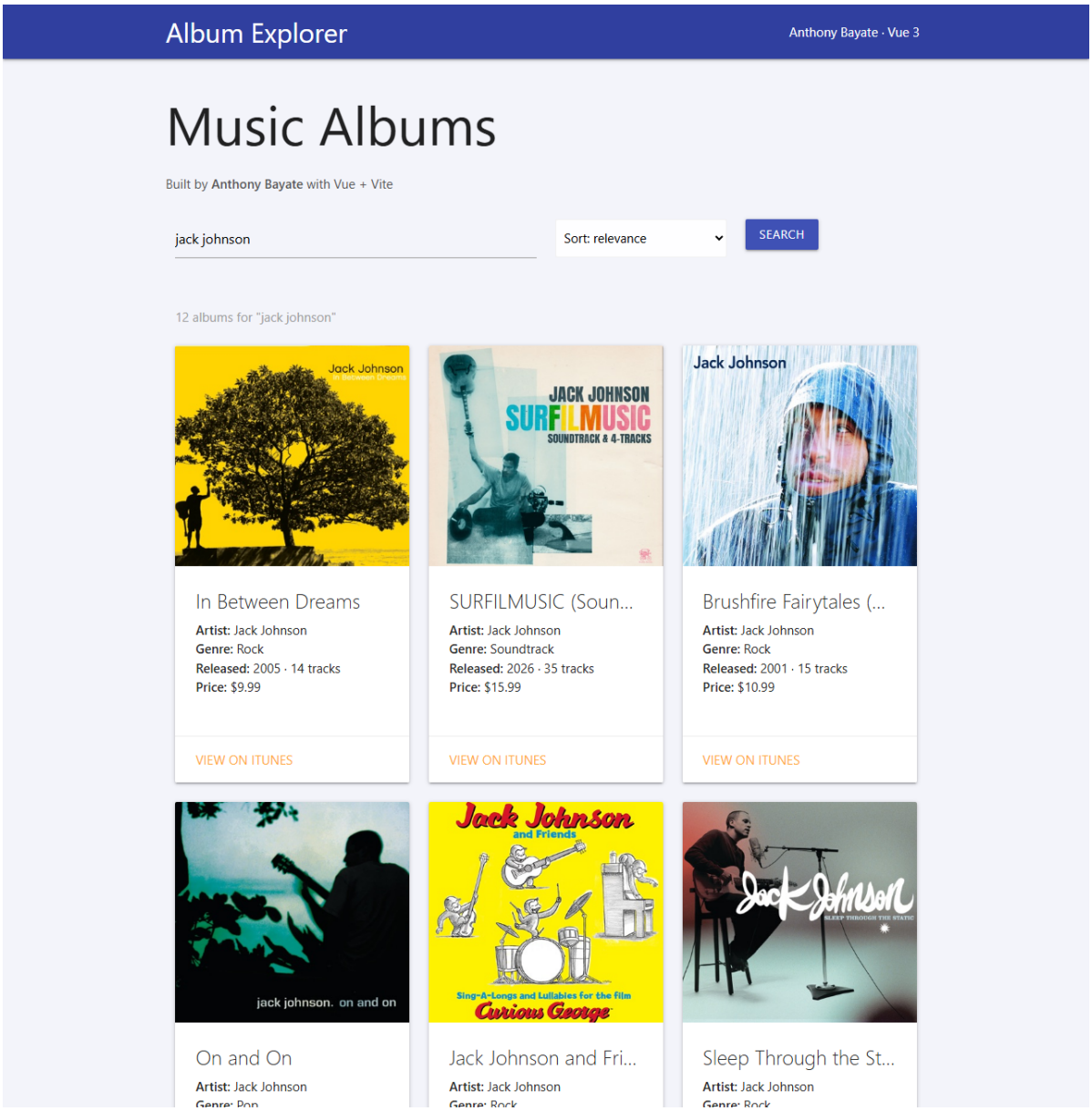
- Search box bound with *v-model*; submitting the form re-runs the fetch with the new artist.
- A *computed()* sorted list (relevance / newest / oldest / most tracks) so sorting never re-hits the API.
- Artwork URL rewritten from 100x100 to 400x400 for sharp images.
- Release year, track count and price on each card; empty-result message; HTTP errors are surfaced.



Default Vue 3 template from Vite



Template with my name added to the GUI



Final customized app (Vue 3) with my name

Task #3. Preact: Met Gallery (Metropolitan Museum Collection API)

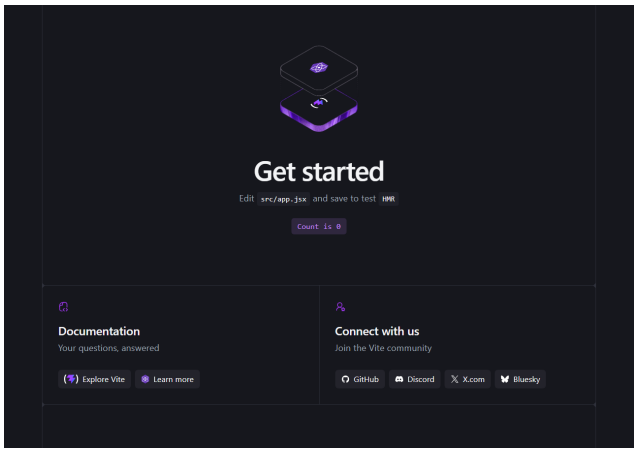
Project folder: *preact/* Dev server: *http://localhost:5103/*

Code discussion (with LLM help)

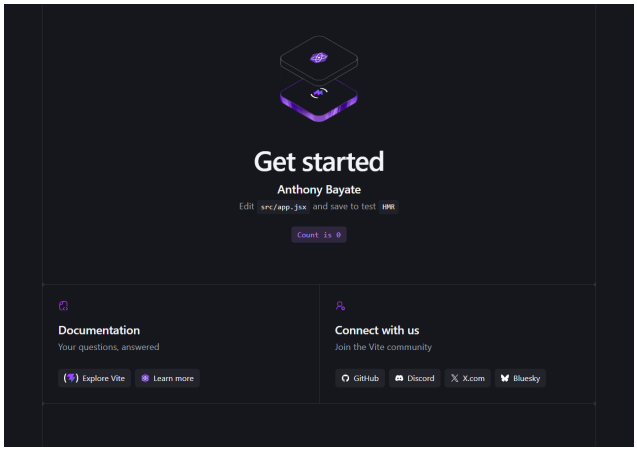
Preact uses the same hooks API as React (*useState*, *useEffect*) in a 3 kB runtime. The starter searches the Met for object IDs, fetches each object in parallel with *Promise.all*, and keeps the ones with images. **Error found:** the Met retired */public/collection/v1/search* on 2026-10-01 (HTTP 410), so the original code shows "Error fetching data". The replacement is the paginated */v1.1/search* endpoint.

Customizations and improvements

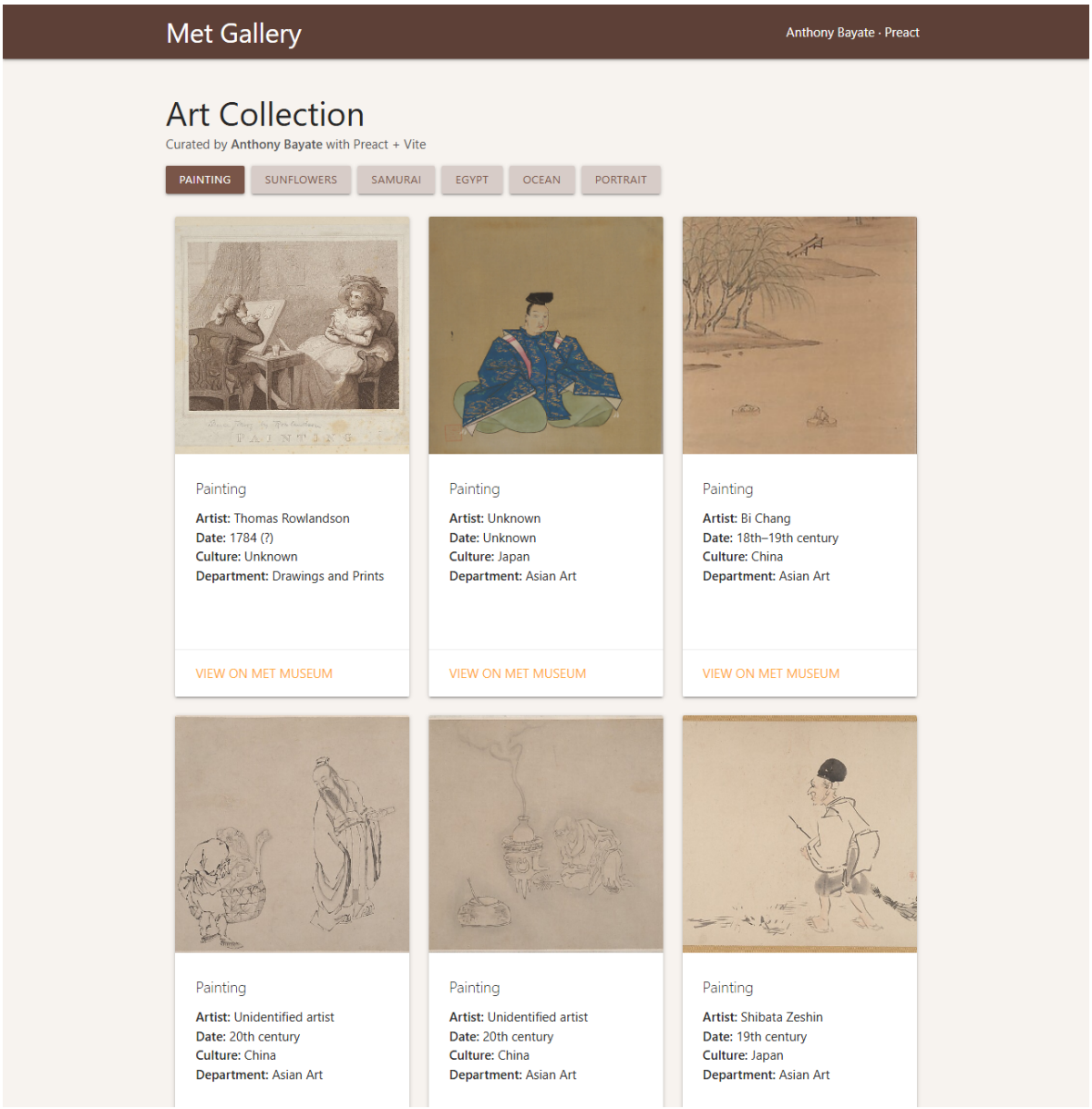
- Switched to */v1.1/search?...&limit=40* (no more downloading every matching ID).
- *Promise.allSettled* so one failed object request doesn't break the whole gallery.
- Topic buttons (painting, sunflowers, samurai, egypt, ...) that re-run the effect via its *[query]* dependency.
- Loading indicator, empty state, and extra fields (date, department).



Default Preact template from Vite



Template with my name added to the GUI



Final customized app (Preact) with my name

Task #4. Lit (Web Components): Fashion Finds (custom element <gif-list>)

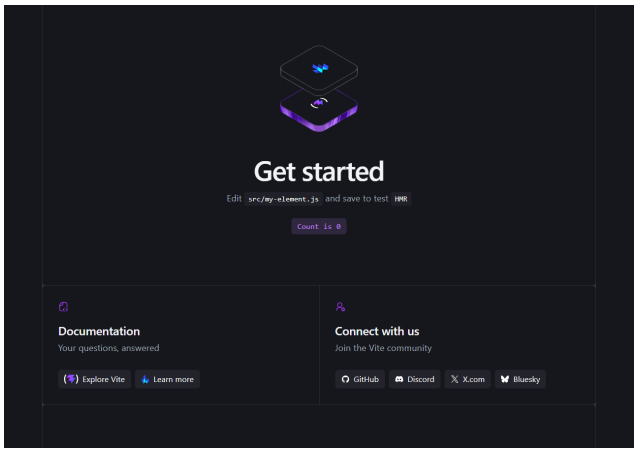
Project folder: *lit/* Dev server: *http://localhost:5104/*

Code discussion (with LLM help)

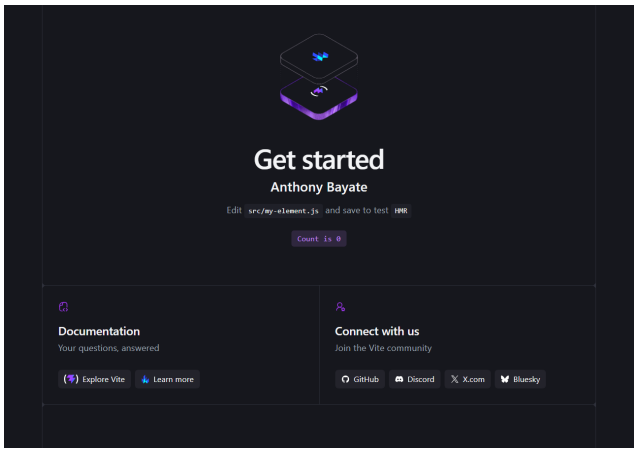
Lit defines a real browser custom element: *static styles* are scoped by the Shadow DOM, *static properties* are reactive, and assigning *this.items* triggers *render()*. **Error found:** the Tenor v1 API now returns *403 "Tenor API is discontinued"*, so the starter renders an empty list. The *node-fetch* install is also unnecessary because browsers have *fetch* built in.

Customizations and improvements

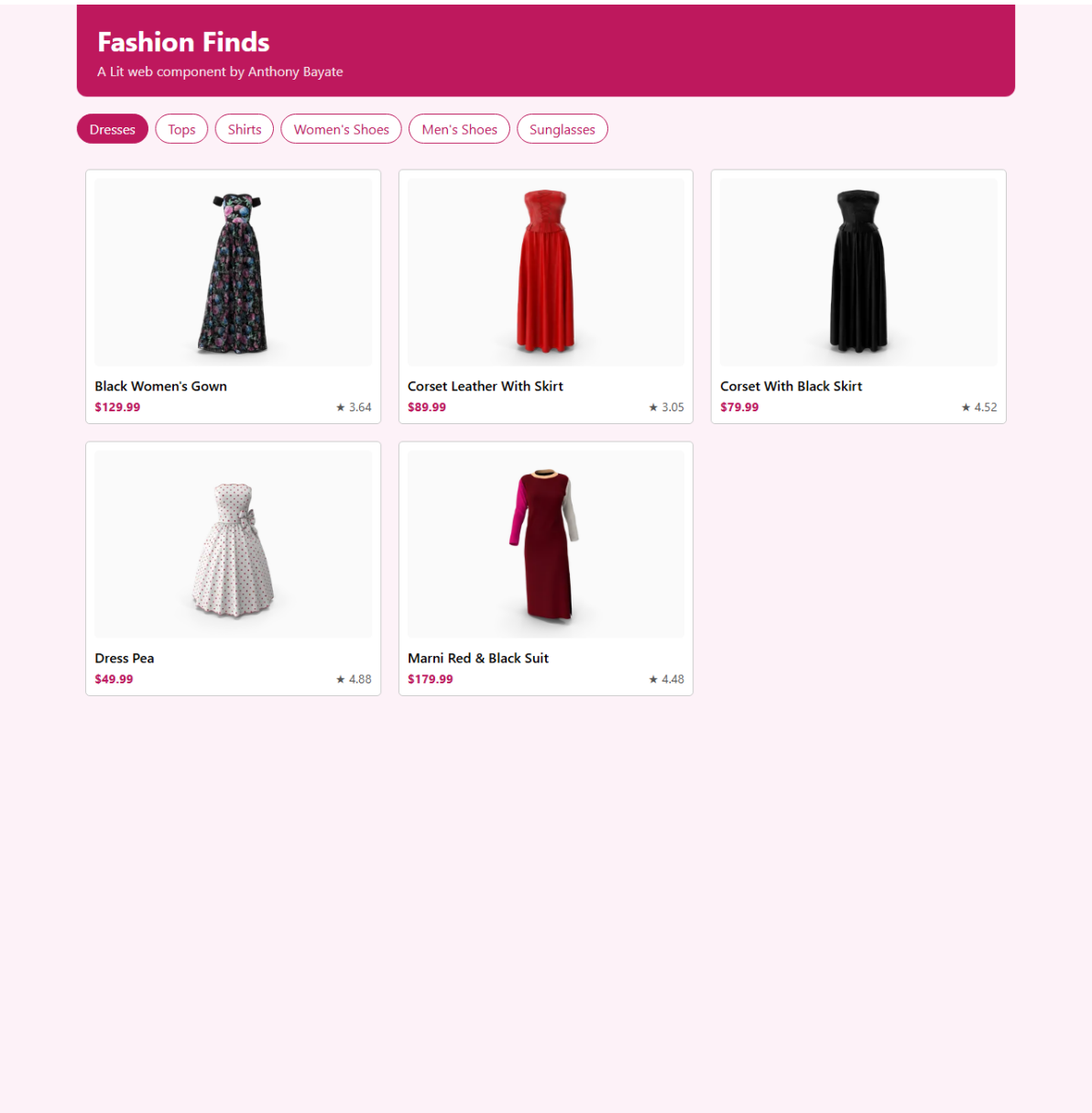
- Data source replaced with DummyJSON's free, keyless fashion product categories (dresses, tops, shirts, shoes, sunglasses).
- Category pill buttons using *@click* and *aria-pressed*; the component re-fetches on change.
- New reactive *owner* attribute, so the name is passed in from HTML: *<gif-list owner="Anthony Bayate">*.
- Loading and error states, price and rating, hover lift, and a single-column layout on small screens.



Default Lit (Web Components) template from Vite



Template with my name added to the GUI



Final customized app (Lit (Web Components)) with my name

Task #5. Svelte 5: NEO Watch (NASA NeoWs API)

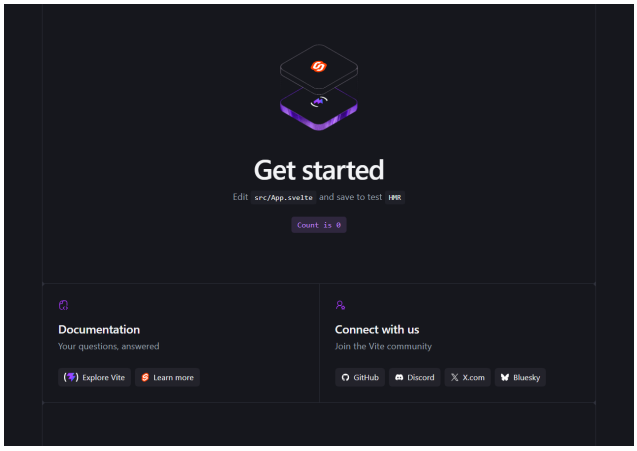
Project folder: *svelte/* Dev server: *http:///localhost:5105/*

Code discussion (with LLM help)

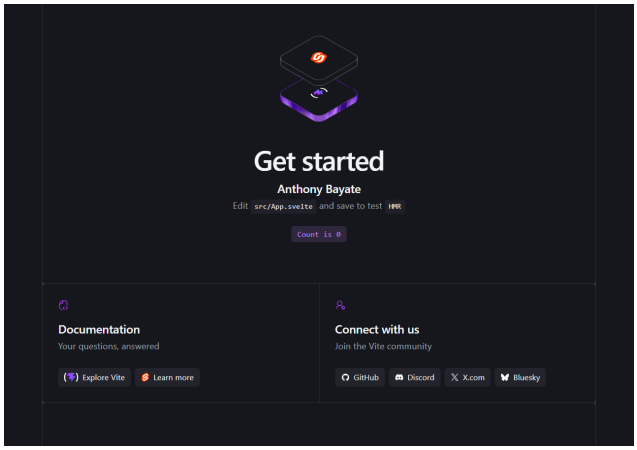
Svelte compiles components to plain JavaScript with no virtual DOM. **Error found:** the lab's *main.js* uses *new App({ target })*, which is Svelte 3/4 syntax; Vite now installs Svelte 5, where components are no longer classes and must be started with *mount(App, { target })*. I kept the template's *mount()* entry point and moved the component to Svelte 5 runes.

Customizations and improvements

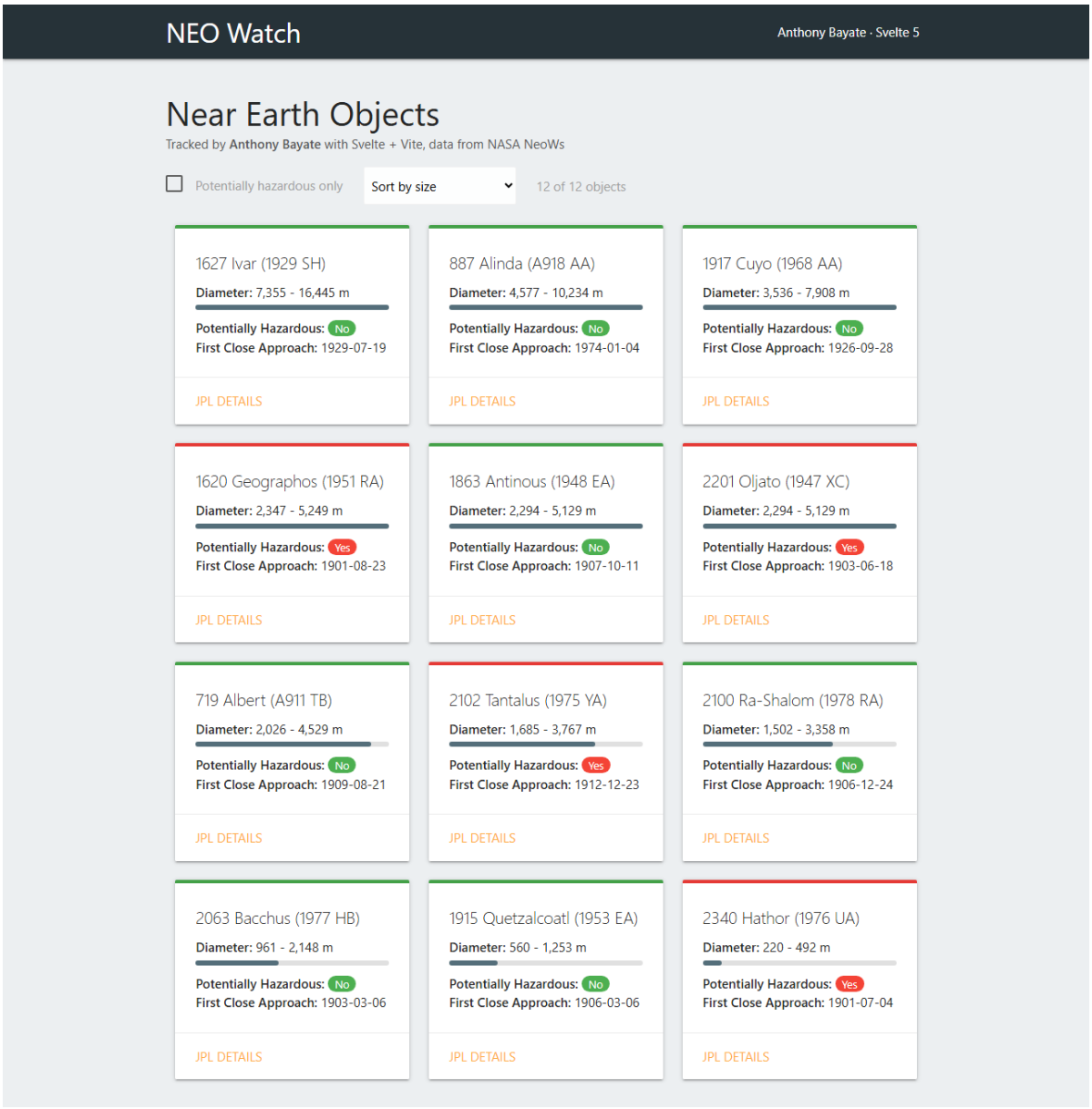
- *$state* for reactive values and *$derived.by* for the filtered/sorted list.
- "Potentially hazardous only" checkbox and sort-by-size/name dropdown with *bind:*.
- Keyed *{#each ... (asteroid.id)}*, rounded diameters, a size bar, and red/green hazard markers.
- Error message when NASA's DEMO_KEY is rate-limited, plus a link to each object's JPL page.



Default Svelte 5 template from Vite



Template with my name added to the GUI



Final customized app (Svelte 5) with my name

Task #6. SolidJS: Toon Shelf (Sample APIs cartoons)

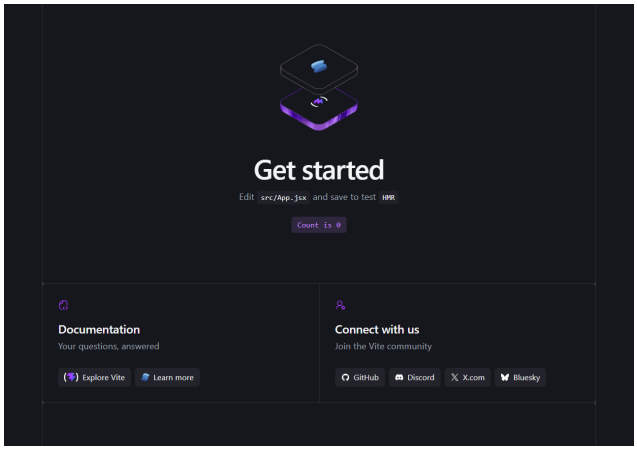
Project folder: *solid/* Dev server: *http:///localhost:5106/*

Code discussion (with LLM help)

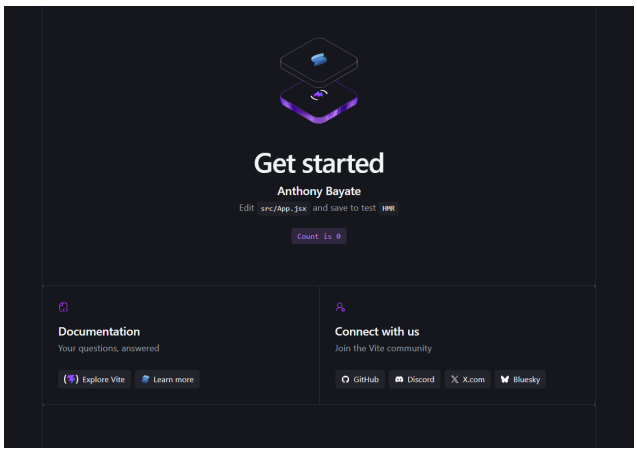
Solid looks like React but its components run only once; *createSignal* values are functions (*cartoons()*) and only the exact DOM nodes that read a signal update. The LLM pointed out that the starter's *.map()* with a *key* prop is a React habit: Solid ignores *key* and recommends <*For*>. Also, *creator* is an array, so it printed run together.

Customizations and improvements

- <*For*> and <*Show*> control-flow components instead of *.map()* and *&&*.
- Live title filter using a *createMemo* derived from the search signal.
- Creators joined with commas, genre chips, episode count and TV rating.
- *onError* fallback: several image hosts block hot-linking, so those cards show a titled placeholder.



Default SolidJS template from Vite



Template with my name added to the GUI

Toon Shelf

Anthony Bayate · SolidJS

Cartoons

Built by **Anthony Bayate** with Solid + Vite

Filter by title...



Gravity Falls

Year: 2012 · 40 episodes · TV-Y7
Creator: Alex Hirsch
Adventure Comedy

Bojack Horseman

Bojack Horseman

Year: 2014 · 77 episodes · TV-MA
Creator: Raphael Bob-Waksberg
Drama Comedy

Steven Universe

Steven Universe

Year: 2013 · 175 episodes · TV-PG
Creator: Rebecca Sugar
Short Adventure



The Adventures of TIN TIN

Year: 2010 · 289 episodes · TV-PG
Creator: Pendleton Ward
Short Action

Pokemon

Pokemon

Year: 1997 · 1131 episodes · TV-Y
Creator: Junichi Masada, Ken Sugimori, Satoshi Tajiri
Adventure Action



Yu-Gi-Oh!

Year: 2000 · 225 episodes · TV-Y
Creator: Kazuki Takashi
Adventure Action



Final customized app (SolidJS) with my name

Task #7. Qwik: Qwik Movies (TVMaze API, Qwik City)

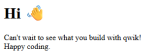
Project folder: *qwik/* Dev server: *http://localhost:5107/*

Code discussion (with LLM help)

Qwik is no longer in the *create-vite* menu, so the project was created with *npm create qwik@latest empty qwik*. Running the lab code as-is crashes the dev server (screenshot below). The LLM and the TypeScript compiler identified two errors: *useClientEffect$* no longer exists in Qwik (renamed to *useVisibleTask$*), and *innerHTML* is not a valid JSX prop (Qwik uses *dangerouslySetInnerHTML*). A third, latent bug: shows with *image: null* would crash *movie.image.medium*.

Customizations and improvements

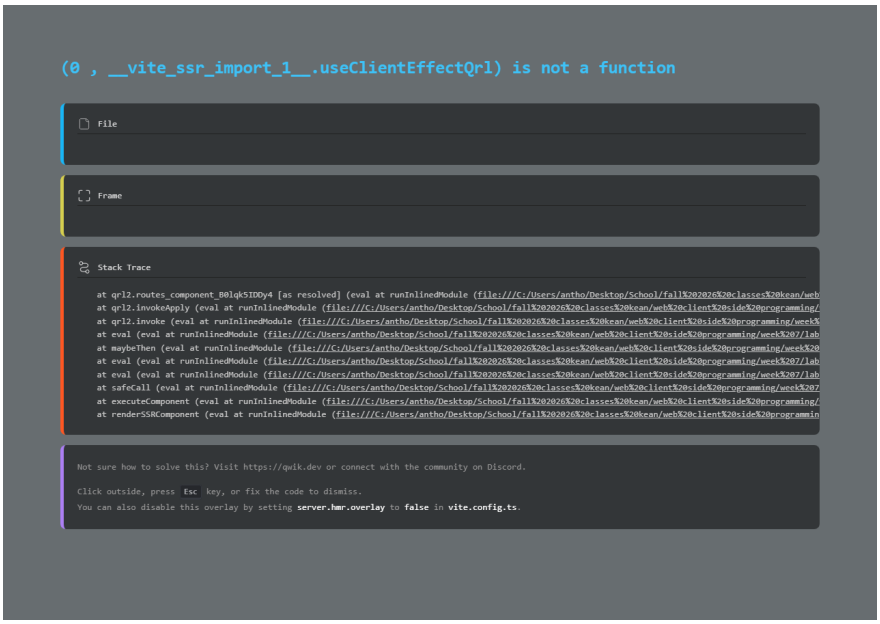
- Data loading moved to *routeLoader$*, which runs on the server, so the HTML arrives already filled in. This is Qwik's resumability idea: no client-side fetch and no hydration.
- *dangerouslySetInnerHTML* for the summary; optional chaining plus a placeholder for missing images.
- Client-side filter with *useSignal* + *useComputed$*; that code is only downloaded when used.
- Rating badges, genres, and a *head* export that sets the page title. *tsc --noEmit* passes.



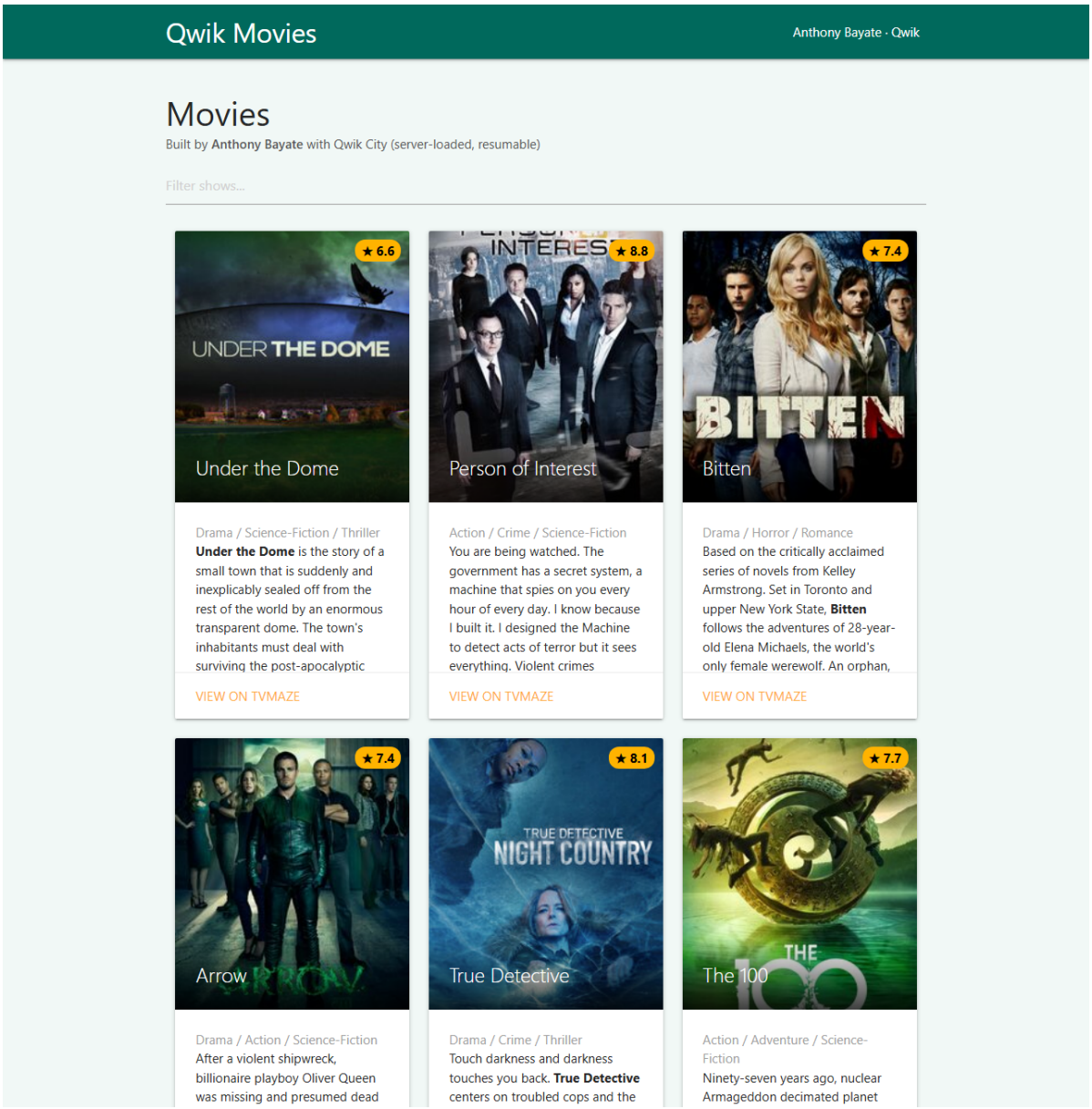
Default Qwik template from Vite



Template with my name added to the GUI



Handout code run as-is: useClientEffect$ is not a function



Final customized app (Qwik) with my name

Takeaways

- **Vanilla** is the most work per feature: every DOM update, escape and state toggle is manual.
- **Vue** felt the most approachable: templates plus *v-model* and *computed* made search and sort short.
- **Preact** is React's API in a much smaller package; my React knowledge carried over directly.
- **Lit** produces a standard custom element that can be dropped into any page or framework.
- **Svelte** had the least boilerplate; runes make it clear which values are reactive.
- **Solid** looks like React but you have to think in signals (*cartoons()*) and use <*For*>.
- **Qwik** was the most different: loading on the server with *routeLoader$* means the page is interactive without hydration.
- Third-party APIs change: two of the six APIs in the handout were retired, so error handling is not optional.